2.680 Lab 09 - Inter-Vehicle Messaging

Inter-vehicle messaging lab material.

Included MIT MOOS-IvP PDF sources:

- lab_class_09_interv_msg.pdf

Lab 09 - Introduction to Constrained Inter-Vehicle Messaging

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications

March 19th, 2026

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview and Objectives

The focus of this lab is an introduction to inter-vehicle messaging, focusing first on messaging in simulation. This will be used in our next lab on autonomous collaborative search. In this lab, two MOOS apps, used for the first time in our labs, will be our focus:

- uFldMessageHandler - A MOOS handler for incoming message from other vehicles.
- uFldNodeComms - A MOOS shoreside arbiter of inter-vehicle messages.

A summary of today's topics:

- Introduction to the uField Toolbox Inter-Vehicle Messaging Apps
- Implement basic messaging in a two-vehicle example
- Range-limited inter-vehicle messaging
- Recovering from an messaging out-of-range situation

2 Preliminaries

Make Sure You Have the Latest Updates

Always make sure you have the latest code:

```
$ cd moos-ivp
$ git pull
$ ./build.sh
```

```
$ cd moos-ivp-2680
$ git pull
$ ./build.sh
```

The latter is especially important as we provide a baseline mission for this lab from the `moos-ivp-2680` tree. The baseline mission was updated very recently.

Make Sure Key Executables are Built and In Your Path

This lab does assume that you have a working MOOS-IvP tree checked out and installed on your computer. To verify this make sure that the following executables are built and findable in your shell path:

```
$ which MOOSDB
/Users/you/moos-ivp/bin/MOOSDB
$ which pHelmIvP
/Users/you/moos-ivp/bin/pHelmIvP
```

If unsuccessful with the above, return to the steps in Lab 1:

http://oceanai.mit.edu/ivpman/labs/machine_setup

Where to Build and Store Lab Missions

As with previous labs, we will use your version of the `moos-ivp-extend` tree. In this tree, there is a missions folder:

```
$ cd moos-ivp-extend
$ ls
CMakeLists.txt  bin/        build.sh*     clean.sh      docs/         missions/
README          build/      data/         lib/          scripts/      src/
```

For each distinct assignment in this lab, there should be a corresponding subdirectory in a `lab_09` sub-directory of the `missions` folder, typically with both a `.moos` and `.bhv` configuration file. See Section 9.1 for the full requested file structure.

Documentation Conventions

To help distinguish between MOOS variables, MOOS configuration parameters, and behavior configuration parameters, we will use the following conventions:

- MOOS variables are rendered in `green`, such as `IVPHELM_STATE`, as well as postings to the `MOOSDB`, such as `DEPLOY=true`.
- MOOS configuration parameters are rendered in `blue`, such as `AppTick=10` and `verbose=true`.
- Behavior parameters are rendered in `brown`, such as `priority=100` and `endflag=RETURN=true`.
- MOOS-IvP applications are rendered in `magenta`, such as `pShare`, or `pHelmIvP`.
- General GNU/Linux commands are represented in `dark purple`, such as `wget`, `mkdir`, or `cd`.

More MOOS / MOOS-IvP Resources

- The slides from class:
  http://oceanai.mit.edu/2.680/docs/2.680-10-intervehicle_messaging_2026.pdf
- The `uFldMessageHandler` documentation
  http://oceanai.mit.edu/ivpman/apps/uFldMessageHandler
- The `uFldNodeComms` documentation
  http://oceanai.mit.edu/ivpman/apps/uFldNodeComms

You can also find your way to the documentation by running the app on the command line with the `--web` or `-w` argument.

3 Inter-Vehicle Messaging

The exercise in today's lab involves inter-vehicle messaging. Since inter-vehicle messaging is a component of later labs, we isolate this issue first with today's lab. The goal will be to construct a two vehicle mission where each vehicle is loitering in a pattern on the west and east side of an operation area respectively. Each vehicle will periodically send the other vehicle a message containing a new latitude (Y value in local coordinates) to shift its loiter pattern.

The primary new modules being used in this lab are the `uFldMessageHandler` and `uFldNodeComms` modules. See the corresponding class notes for a description:

http://oceanai.mit.edu/2.680/docs/2.680-10-intervehicle_messaging_2026.pdf

And see the MOOS-IvP Tools documentation available on the course website for full documentation. The figure below from today's lecture illustrates the basic setup. The focus today is on these two modules.

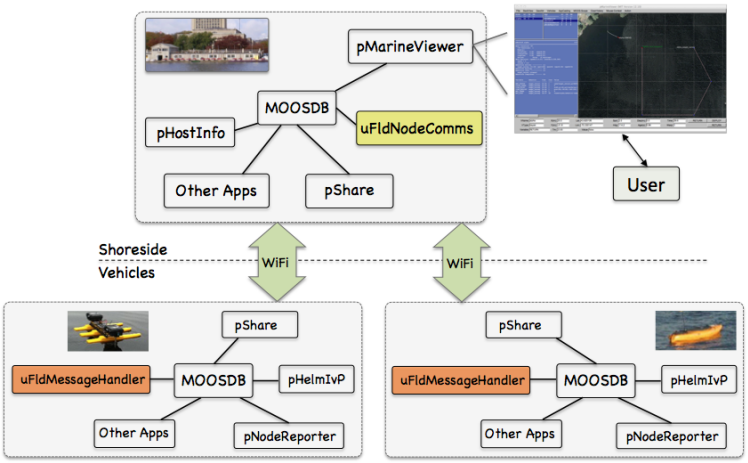

Figure 1: **The uFldMessageHandler and uFldNodeComms Modules:** The `uFldMessageHandler` runs on each vehicle and parses incoming `NODE_MESSAGE` postings. The `uFldNodeComms` module runs on the shoreside and routes messages to their destination vehicle(s).

4 Assignment 1 (self check off) - Charlie Dana Baseline Mission

Our first step is to get and run the baseline mission for this lab. In your `moos-ivp-extend/missions` folder, create a new sub-folder `lab_09`. Make sure you update (git pull) the `moos-ivp-2680` repo first. Then copy the baseline mission folder into your new `lab_09` folder.

```
$ cd moos-ivp-2680/missions
$ cp -rp lab_09_charlie_dana_baseline moos-ivp-extend/missions/lab_09/charlie_dana_baseline
```

This mission is configured for two vehicles, `charlie` and `dana`, each doing a simple loiter mission with the ability to return and station-keep at any time. It should contain nothing you have not seen before in prior labs. It should be launchable with:

```
$ cd charlie_dana_baseline
$ ./launch.sh 15
```

It is our starting point for this lab, and should look something like the video posted at:

Figure 2: A simple two-vehicle baseline mission with both vehicles loitering at a fixed location.

5 Assignment 2 (self check off) - Charlie Dana NodeComms Mission

The next step is to augment your baseline mission to support inter-vehicle communications. The first key addition is the module uFldNodeComms which runs on the shoreside and accepts node reports from all connected vehicles. To get started, make a copy of your baseline mission into a folder called charlie_dana_nodecomms.

```
$ cd moos-ivp-extend/missions/lab_09
$ cp -rp charlie_dana_baseline  charlie_dana_nodecomms
```

The first step is to add uFldNodeComms to your pAntler configuration block in meta_shoreside.moos:

```
ProcessConfig = ANTLER
{
  MSBetweenLaunches = 100

  Run = MOOSDB          @ NewConsole = false
  Run = pMarineViewer   @ NewConsole = false
  Run = pLogger         @ NewConsole = false
  Run = uXMS            @ NewConsole = false
  Run = uProcessWatch   @ NewConsole = false
  Run = pShare          @ NewConsole = false
  Run = pHostInfo       @ NewConsole = false
  Run = uFldShoreBroker @ NewConsole = false
  Run = uFldNodeComms   @ NewConsole = false   <-- Add this line (but not this comment)
}
```

The next step is to add a uFldNodeComms configuration block also to the meta_shoreside.moos configuration file. You are encouraged to take a read through the uFldNodeComms documentation to know more about the below parameters, but here is an example configuration block:

```
ProcessConfig = uFldNodeComms
{
  AppTick    = 2
  CommsTick  = 2

  comms_range      = 120
  critical_range   = 30
  min_msg_interval = 15
  max_msg_length   = 1000

  view_node_rpt_pulses = true
}
```

The key parameter initially is the comms_range parameter. Setting this value here to 120 means that any two vehicles must be within 120 meters of one another for an inter-vehicle message to go through. The other parameters limit the message frequency and length, and the last parameter affects whether visual artifacts are also produced by uFldNodeComms when it is running. Once you have these changes in place, re-launch the mission.

```
$ cd moos-ivp-extend/missions/lab_09/charlie_dana_nodecomms
$ ./launch.sh 15
```

It should look something like the video posted at:

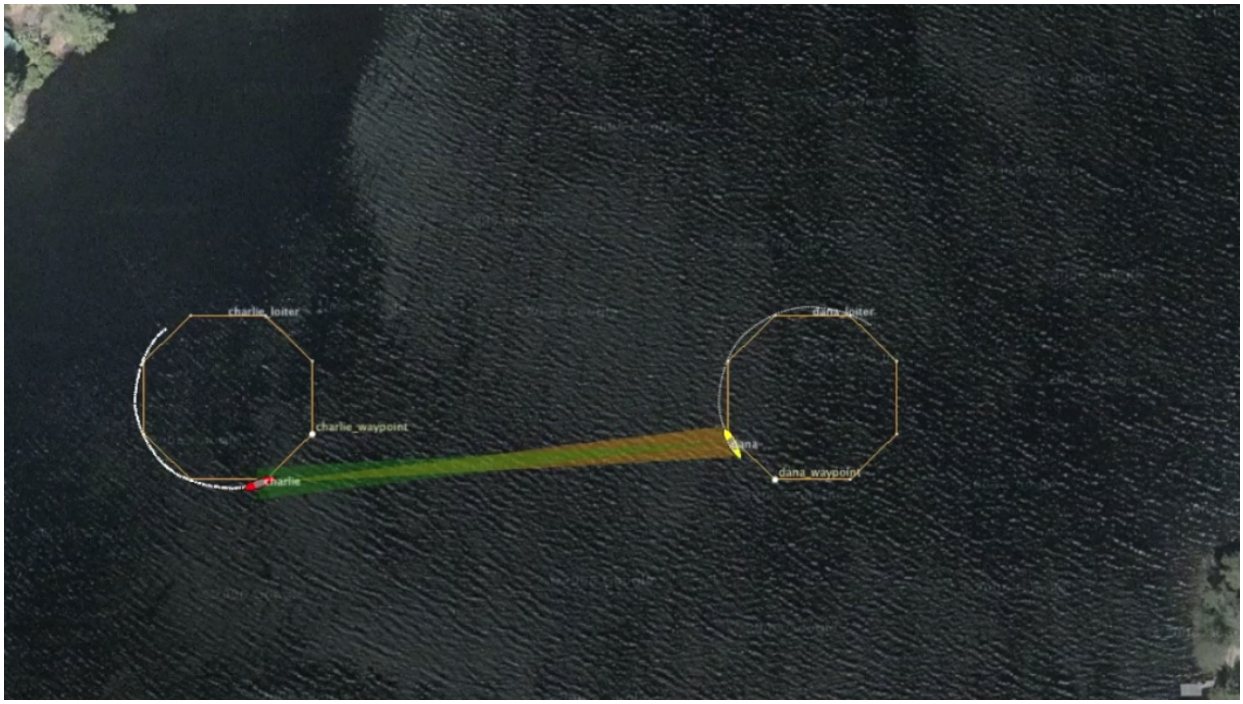

Figure 3: A simple two-vehicle baseline mission with both vehicles loitering at a fixed location, and inter-vehicle communications supported with uFldNodeComms running. The colored cones between vehicles indicate the vehicles are within comms range of each other.

6 Assignment 3 (check off) - The Charlie Dana Message Mission

The next step is to augment your mission to test inter-vehicle communications. The first key addition is the module uFldMessageHandler which runs on each of the vehicles and accepts node messages from other vehicles. To get started, you can make a copy of your previous mission into a folder called charlie_dana_message.

```
$ cd moos-ivp-extend/missions/lab_09
$ cp -rp charlie_dana_nodecomms  charlie_dana_message
```

The first step is to add a uFldMessageHandler to your pAntler configuration block in meta_vehicle.moos:

```
ProcessConfig = ANTLER
{
  MSBetweenLaunches = 100

  Run = MOOSDB            @ NewConsole = false
  Run = uProcessWatch     @ NewConsole = false
  Run = pShare            @ NewConsole = false
  Run = uSimMarineV22     @ NewConsole = false
  Run = pLogger           @ NewConsole = false
  Run = pNodeReporter     @ NewConsole = false
  Run = pMarinePIDV22     @ NewConsole = false
  Run = pHelmIvP          @ NewConsole = false
  Run = pHostInfo         @ NewConsole = false
  Run = uFldNodeBroker    @ NewConsole = false
  Run = uFldMessageHandler @  NewConsole = false   <-- Add this line, but not this comment
}
```

The next step is to add a uFldMessageHandler configuration block also to the meta_vehicle.moos configuration file. You are encouraged to take a read through the uFldMessageHandler if you have time, but here is an example configuration block:

```
ProcessConfig = uFldMessageHandler
{
  AppTick    = 2
  CommsTick  = 2

  strict_addressing = false
}
```

The strict_addressing configuration parameter indicates whether incoming messages need to be addressed strictly to the named vehicle or whether messages sent to "all" vehicles will also be accepted. We leave it set to false so both kinds of messages are accepted.

The last step is to share outgoing vehicle messages, generated on a vehicle, to the shoreside for distribution to other vehicles. To do this, the uFldNodeBroker configuration block in meta_vehicle.moos needs to be augmented with the below line (if it isn't there already).

```
  bridge = src=VIEW_POINT
  bridge = src=VIEW_SEGLIST
  bridge = src=APPCAST
  bridge = src=NODE_REPORT_LOCAL, alias=NODE_REPORT
  bridge = src=NODE_MESSAGE_LOCAL,  alias=NODE_MESSAGE  <-- Add this line, but not this comment
```

Once you have these changes in place, re-launch the mission.

```
$ cd moos-ivp-extend/missions/lab_09/charlie_dana_message
$ ./launch.sh 15
```

At this point, the vehicles are ready to send messages between each other, but nothing is happening yet.

How does charlie, meaning to send a message to dana, initiate this message? Charlie posts to its MOOSDB, an outgoing message in the variable NODE_MESSAGE_LOCAL. As described in the uFldMessageHandler documentation, this message has four main components:

- src_node: The name of the vehicle originating the message, in this case charlie.
- dest_node: The name of the vehicle for which the message is intended to be sent, in this case dana.
- var_name: Name of the MOOS variable we want written in the receiving vehicle's MOOS community. In this case the variable name is UP_LOITER because each vehicle is running with a loiter behavior configured with updates=UP_LOITER to accept dynamic behavior parameter changes. See the loiter configuration block inside meta_vehicle.bhv.
- string_val: The contents of the MOOS variable when posted in the receiving vehicle's MOOS community. In this case we use string_var to indicate we are posting a string rather than a double. And in this case we are passing a new loiter y-position to the receiving vehicle. The parameter ycenter_assign is a defined parameter for the loiter behavior, meant just for these situations.

So, the outgoing node message will have the following four components:

- src_node=charlie
- dest_node=dana
- var_name=UP_LOITER
- string_val=ycenter_assign=-50

If you'd like to poke the loiter change *directly* into dana, try this:

```
$ uPokeDB targ_dana.moos UP_LOITER=ycenter_assign=-50
```

To simulate a message going from charlie to dana (the goal/criteria of this assignment), poke charlie with an outgoing message formed as above:

```
$ uPokeDB targ_charlie.moos NODE_MESSAGE_LOCAL="src_node=charlie,dest_node=dana,
          var_name=UP_LOITER,string_val=ycenter_assign=-50"
```

Keep a command window open and try poking the above message alternating between -50 and -100 for the new y-loiter position for dana. It should look something like the video posted at:

Figure 4: A simple two-vehicle baseline mission with both vehicles loitering at a fixed location, and inter-vehicle communications supported with uFldNodeComms running. A message is periodically sent from charlie to dana, indicated by the brief white comms cone between the two vehicles. When they are outside of comms range, no colored comms cones rendered, there are no inter-vehicle messages.

7 Assignment 4 (check off) - The Charlie Dana ReAssign Mission

We augment the previous mission where messages were sent by poking the MOOSDB on the sender vehicle. In this mission, messages originate from both vehicles automatically with a simple timer script to re-assign the location of the other vehicle. To get started, make a copy of your previous mission into a folder called `charlie_dana_reassign`.

```
$ cd moos-ivp-extend/missions/lab_09
$ cp -rp charlie_dana_message  charlie_dana_reassign
```

The first step is to add a timer script to your vehicles. The timer script essentially does the same thing you did manually by poking the `MOOSDB` in the Charlie Dana Message mission. A script should be added to your mission by:

- Add `uTimerScript` to the `Antler` block in your `meta_vehicle.moos` file
- Add the below config `uTimerScript` config block to the `meta_vehicle.moos` file (or create and use a new plug if you prefer).

```
ProcessConfig = uTimerScript
{
  AppTick    = 2
  CommsTick  = 2

  condition  = DEPLOY=true
  randvar    = varname=YPOS, min=-125, max=0, key=at_reset

  // THE BELOW EVENT ALL ON ONE LINE IN THE ACTUAL MOOS FILE

  event      = var=NODE_MESSAGE_LOCAL, val="src_node=$(VNAME),dest_node=all,
               var_name=UP_LOITER,string_val=ycenter_assign=$[YPOS]", time=60:90

  reset_max = nolimit
  reset_time = all-posted
}
```

With this script in place, a re-assign message will be periodically sent to the other vehicle. Actually with the syntax above, the message is sent to ”all” other vehicles, but in our simple example there is only one other vehicle, and `uFldMessageHandler` and `uFldNodeComms` prevent messages from being sent back to the sender. The additional condition of `DEPLOY=true` is meant to prevent the re-assigning from happening until the vehicles are deployed.

It should look something like the video posted at:

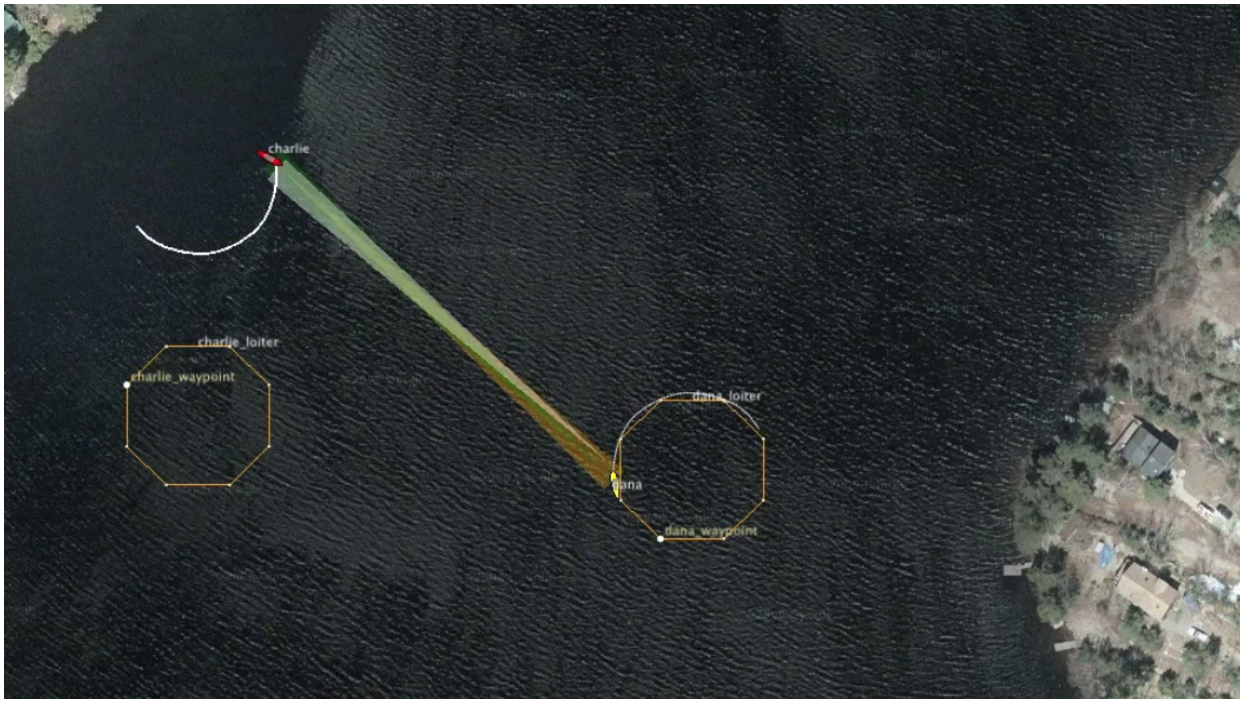

Figure 5: The same simple two-vehicle baseline mission with both vehicles loitering, initially at a fixed location. Inter-vehicle communications is supported with `uFldNodeComms` and `uFldMessageHandler` running. A message is periodically sent from charlie to dana, and vice versa, indicated by the brief white comms cone between the two vehicles. Each message re-assigns the other vehicle to a different loiter location either a little bit north or south of its present position. When they are outside of comms range, with no colored comms cones rendered, there are no inter-vehicle messages.

8 Assignment 5 - The Charlie Dana Recover Mission

In the previous mission, Charlie Dana Re-assign, note that it is possible that the two vehicles re-assign each other to be perpetually out of comms range with one another. Once they are out of comms range, they never change their loiter position to get back into comms range. You may even notice that this almost happens right at the start of the example video in Figure 5. Charlie is loitering in the north-west, and dana in the south-east but dana just manages to squeeze out a message to charlie.

The assignment is to come up with ways to recover from this scenario of being outside of comms range. How can it be detected? What can be done to recover? One way to recover is to have both vehicles return home, and re-connect this way. But can you do better? Can this be done without writing a new MOOS App? (there is no particular answer we're looking for. But there are perhaps several ways to mitigate this problem.)

Note that you can make the problem situation a bit more likely to occur, for the sake of efficient testing, if you shrink the comms range by reducing `comms_range=120` in the `uFldNodeComms` configuration block. Or you can expand the possible loiter locations to the north and south by adjusting the `randvar` parameter in the `uTimerScript` configuration on the vehicles.

There are multiple ways to accomplish the desired results of this lab. For one of the ways, a tool that you might find handy is the `msg_flag` feature of `uFldMessageHandler` which posts a user-configurable flag (MOOS variable and value pair) upon the receipt of each incoming message. See the Event Flags section of the `uFldMessageHandler` app:

```
$ uFldMessagHandler --web
```

9 Instructions for File Organization of Assignments

9.1 Requested File Structure

Here is the requested file structure:

```
moos-ivp-extend/
  missions/
    lab_09/
      charlie_dana_baseline/      // Assignment 1 - self check off
      charlie_dana_nodecomms/     // Assignment 2 - self check off
      charlie_dana_message/       // Assignment 3 - check off
      charlie_dana_reassign/      // Assignment 4 - check off
      charlie_dana_recover/       // Assignment 5 - check off
```

9.2 Due Date

This lab should be completed to be checked off in lab on Tuesday March 31st, 2026.